XCS224N Notes

Christopher Stewart, NLP with Deep Learning, Spring 2026

This document captures high-level ideas from Stanford Engineering's NLP with Deep Learning class (XCS224N) in Spring 2026. The document is organized by module. The course contains 10 modules and 5 assignments.

1 Exploring Word Embeddings

1.1 Word Embeddings

A **word embedding** is a dense vector representation of a word, typically in 50–300 dimensions. Unlike one-hot encodings (which are sparse and treat every word as equally different), embeddings encode semantic meaning: words with similar meanings end up close together in vector space. Embeddings are the input layer for nearly every NLP model. Whether you're doing classification, generation, or retrieval, the quality of your embeddings directly affects downstream performance. Pre-trained embeddings (Word2Vec, GloVe, FastText) can be used off-the-shelf or fine-tuned.

1.2 Two Approaches to Building Embeddings

1.2.1 Count-Based: Co-occurrence Matrices

Build a matrix $\mathbf{M}$ where entry M_{ij} counts how often word i appears near word j within a context window of size n. Words that appear in similar contexts get similar row vectors. **Key properties:** The matrix is symmetric ($M_{ij} = M_{ji}$). It's very large (vocabulary size squared) and sparse. Dimensionality reduction via techniques like SVD are used to make it feasible to use such approaches. **Context window:** For a window size of n, you look at n words to the left and n to the right of each center word. Words near the edges of a document have smaller windows.

1.2.2 Prediction-Based: Word2Vec

Instead of counting co-occurrences, Word2Vec trains a shallow neural network to predict context words from a center word (Skip-gram) or vice versa (CBOW). The learned weight matrix becomes the embedding. **Key differences from co-occurrence:** Word2Vec is trained on very large corpora (billions of words), captures more nuanced relationships, and produces dense vectors directly without needing SVD. However, it produces a single vector per word, so polysemous words (multiple meanings) are blended into one representation, one that is typically dominated by the most frequent meaning.

1.3 SVD and Dimensionality Reduction

Singular Value Decomposition factors any matrix $\mathbf{A} \in \mathbb{R}^{n \times d}$ into three pieces:

$$\mathbf{A} = \mathbf{U}\mathbf{D}\mathbf{V}^T$$

$\mathbf{U}$ contains the left singular vectors (one per row/word), $\mathbf{D}$ is diagonal with singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r$ measuring each component's importance, and $\mathbf{V}$ contains the right singular vectors.

The outer product form shows $\mathbf{A}$ as a sum of layers: $\mathbf{A} = \sum_{i=1}^{r} \sigma_i \mathbf{u}_i \mathbf{v}_i^T$. Each layer $\sigma_i \mathbf{u}_i \mathbf{v}_i^T$ is a rank-1 matrix capturing one pattern. The first layer captures the most important pattern, the second adds the next, and so on. **Truncated SVD** keeps only the top k layers: $\mathbf{A}_k = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^T = \mathbf{U}_k \mathbf{D}_k \mathbf{V}_k^T$. This is the best rank-$k$ approximation of $\mathbf{A}$ (minimizes Frobenius norm error). In practice, reducing a co-occurrence matrix from thousands of dimensions to $k = 100$–300 produces useful word vectors. Implementing SVD from scratch is uncommon. `sklearn.decomposition.TruncatedSVD` handles it. The key insight is understanding *why* it works: it finds the directions of maximum variance in the data and projects onto them.

1.4 Measuring Word Similarity

Cosine similarity measures the angle between two word vectors, ignoring magnitude:

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|}$$

Ranges from -1 (opposite) to 1 (identical direction). This is preferred over Euclidean distance because it's invariant to vector length, i.e. a word that appears 10x more often shouldn't be considered 10x more "meaningful." **Cosine distance** $= 1 - \cos(\mathbf{u}, \mathbf{v})$. Smaller distance means more similar. **Surprising finding:** Antonyms often have *small* cosine distance (high similarity) because they appear in very similar contexts ("the weather is hot/cold", "the movie was good/bad"). Word vectors capture *distributional* similarity, not semantic opposition.

1.5 Word Analogies

Word embeddings capture relational structure. The classic example:

$$\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$$

This works because the vector from "man" to "king" encodes the concept of "royalty," and adding it to "woman" lands near "queen." Analogies are solved by finding the word whose vector is closest (by cosine similarity) to the target expression. **Limitations:** Analogies work well for certain relationships (gender, country-capital, tense) but fail for others. They also reflect biases in the training corpus (e.g., associating certain professions with genders).

1.6 Bias in Word Vectors

Word vectors inherit biases from their training data. If the corpus associates certain races, genders, or groups with particular properties, those associations are encoded in the vectors. Any downstream model using biased embeddings will propagate those biases. Debiasing techniques exist but are imperfect. Being aware of this is essential when deploying NLP systems in production.

1.7 Key Takeaways for Later Modules

- Nearly every NLP architecture starts by converting words to vectors.
- Pre-trained embeddings (Word2Vec, GloVe) give you a strong starting point; fine-tuning adapts them to your task.
- Contextual embeddings like BERT and GPT, covered in later modules, solve the polysemy problem by giving each *occurrence* of a word a different vector based on its context.
- SVD and matrix factorization ideas recur in recommender systems, topic models, and understanding attention mechanisms.
- The "distributional hypothesis" (words are defined by their context) underlies both co-occurrence and Word2Vec, and later, transformer-based models.

Math Foundations Quick Reference Vectors and Matrices: A vector $\mathbf{u} \in \mathbb{R}^n$ is a list of n numbers. A matrix $\mathbf{A} \in \mathbb{R}^{n \times d}$ has n rows and d columns. The transpose $\mathbf{A}^T$ flips rows and columns. **Outer Product:** A column vector times a row vector: $\mathbf{u}\mathbf{v}^T$ produces an $n \times d$ matrix. Each entry $(i, j) = u_i \cdot v_j$. This creates a rank-1 matrix, or a single "pattern." **Orthonormality:** Vectors $\mathbf{v}_1, \ldots, \mathbf{v}_r$ are orthonormal if $\mathbf{v}_i^T \mathbf{v}_j = 0$ when $i \neq j$ (perpendicular) and $\mathbf{v}_i^T \mathbf{v}_i = 1$ (unit length). In SVD, both $\mathbf{U}$ and $\mathbf{V}$ have orthonormal columns. This property is what makes terms cancel cleanly when you multiply by a single $\mathbf{v}_i$. **Projection:** The projection of a vector $\mathbf{a}$ onto a subspace spanned by orthonormal $\mathbf{v}_1, \ldots, \mathbf{v}_k$ is $\sum_{i=1}^{k} (\mathbf{a}^T \mathbf{v}_i)\mathbf{v}_i$. In matrix form: $\mathbf{A}\mathbf{V}_k\mathbf{V}_k^T$ projects all rows of $\mathbf{A}$ at once. Truncated SVD *is* this projection. **Frobenius Norm:** $\|\mathbf{M}\|_F = \sqrt{\sum_{i,j} M_{ij}^2}$, the "size" of a matrix, analogous to vector length. Truncated SVD minimizes $\|\mathbf{A} - \mathbf{B}\|_F$ over all rank-k matrices $\mathbf{B}$. **Diagonal Matrix Multiplication:** Multiplying $\mathbf{UD}$ where $\mathbf{D}$ is diagonal just scales each column of $\mathbf{U}$: column i gets multiplied by σ_i. No mixing of columns occurs.

2 Word Vectors and Word2Vec

2.1 Core Activation Functions

Sigmoid. Maps any real number to $(0, 1)$. Used for binary gates and as a building block in many architectures.

$$\sigma(x) = \frac{1}{1 + e^{-x}} \qquad \text{Derivative: } \sigma'(x) = \sigma(x)(1 - \sigma(x))$$

The derivative being expressible in terms of σ itself is useful: once you've computed the forward pass, the backward pass is nearly free. Numerically, when x is very negative, compute $\sigma(x) = \frac{e^x}{1+e^x}$ instead to avoid overflow in e^{-x}. **Softmax.** Generalizes sigmoid to V classes. Takes a vector of raw scores (called *logits*) $\mathbf{z} \in \mathbb{R}^V$ and converts it into a probability distribution. The formula for the probability assigned to word w is:

$$\hat{y}_w = \frac{\exp(z_w)}{\sum_{w'=1}^{V} \exp(z_{w'})}$$

Here's how to read this, step by step. The numerator $\exp(z_w)$ means "take $e \approx 2.718$ and raise it to the power of the score for word w." This turns every score, positive, negative, or zero, into a positive number. The denominator does the same thing for *every* word in the vocabulary and sums them up. Dividing the numerator by that sum guarantees all the probabilities are positive and add to 1. Exponentiation amplifies differences: a score twice as large gets disproportionately more probability, so the highest-scored word dominates. In practice, subtract $\max(\mathbf{z})$ from all entries before exponentiating for numerical stability (this doesn't change the result because it cancels out in the fraction). **Example:** Suppose we have $V = 3$ words with logits $\mathbf{z} = [2.0,\ 1.0,\ 0.5]$.

1. Exponentiate each score: $\exp(2.0) = 7.389, \quad \exp(1.0) = 2.718, \quad \exp(0.5) = 1.649$.
2. Sum the results: $7.389 + 2.718 + 1.649 = 11.756$.
3. Divide each by the sum:

$$\hat{y}_1 = \frac{7.389}{11.756} = 0.629, \quad \hat{y}_2 = \frac{2.718}{11.756} = 0.231, \quad \hat{y}_3 = \frac{1.649}{11.756} = 0.140$$

So the model assigns 62.9% probability to word 1, 23.1% to word 2, and 14.0% to word 3. Notice that $0.629 + 0.231 + 0.140 = 1.0$.

2.2 Cross-Entropy Loss

The standard loss for classification. Given a true one-hot distribution $\mathbf{y}$ and predicted distribution $\hat{\mathbf{y}}$:

$$J = -\sum_w y_w \log(\hat{y}_w) = -\log(\hat{y}_o)$$

where o is the index of the correct class. The simplification follows because $\mathbf{y}$ is one-hot ($y_o = 1$, all others 0). Intuition: penalizes the model by $-\log$ of the probability it assigned to the correct answer. Confident and correct $\rightarrow$ low loss. Confident and wrong $\rightarrow$ very high loss.

2.3 The Error Signal Pattern

A recurring motif in deep learning is that when you combine softmax with cross-entropy loss, the gradient takes the elegant form $(\hat{\mathbf{y}} - \mathbf{y})$. This is the *error signal*, i.e. the difference between what the model predicted and the truth. For the correct class, the entry is $(\hat{y}_o - 1) < 0$ (model didn't assign enough probability). For all other classes, the entry is $\hat{y}_w > 0$ (model assigned too much). This pattern appears in logistic regression, word2vec, and the output layers of most neural networks.

2.4 Word2Vec: Skip-Gram Model

Core idea: Learn word vectors by training a model to predict context words from a center word. Each word has two vectors: a center vector $\mathbf{v}_c$ (used when the word is the center) and an outside vector $\mathbf{u}_w$ (used when it's a context word). **Prediction:** The probability of outside word o given center word c:

$$P(o \mid c) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)}$$

The dot product $\mathbf{u}_o^\top \mathbf{v}_c$ measures alignment between vectors. Higher alignment $\rightarrow$ higher predicted probability. **Training:** For each (center, context) pair, compute cross-entropy loss and update both vectors via SGD. The skip-gram treats each context word independently and sums losses across all context positions. **Gradients:** Both gradients flow from the same error signal:

$$\frac{\partial J}{\partial \mathbf{v}_c} = \mathbf{U}^\top(\hat{\mathbf{y}} - \mathbf{y}) \qquad \frac{\partial J}{\partial \mathbf{U}} = (\hat{\mathbf{y}} - \mathbf{y})\,\mathbf{v}_c^\top$$

2.5 Negative Sampling

Naive softmax requires summing over the entire vocabulary for every training step, which is prohibitively expensive for large vocabularies (V can be 10^5–10^6). Negative sampling approximates this by only considering the true outside word and K randomly sampled "negative" words:

$$J = -\log \sigma(\mathbf{u}_o^\top \mathbf{v}_c) - \sum_{k=1}^{K} \log \sigma(-\mathbf{u}_k^\top \mathbf{v}_c)$$

First term: push $\mathbf{v}_c$ toward $\mathbf{u}_o$ (the true context word). Remaining terms: push $\mathbf{v}_c$ away from K random words. Computational cost drops from $O(V)$ to $O(K)$ per training step, with $K = 5$–20 typical in practice.

2.6 Stochastic Gradient Descent

The workhorse optimization algorithm. At each step, estimate the gradient from a small batch and update:

$$\mathbf{x} \leftarrow \mathbf{x} - \eta\, \nabla_{\mathbf{x}} J$$

where η is the learning rate. Key practical considerations: (1) learning rate annealing (reduce η over time to converge more precisely); (2) the gradient estimate is noisy but unbiased, which actually helps escape local minima; (3) modern variants (Adam, AdaGrad) adapt η per-parameter, but vanilla SGD remains competitive with proper tuning.

2.7 Engineering Takeaways

Softmax + cross-entropy is the default output layer for classification. The clean $(\hat{\mathbf{y}} - \mathbf{y})$ gradient makes implementation straightforward and numerically stable. *Negative sampling* exemplifies a broader principle: when an exact computation is too expensive, sample a few "contrastive" examples instead. This idea recurs in contrastive learning, noise-contrastive estimation, and retrieval-augmented generation. *Dot-product similarity* ($\mathbf{u}^\top \mathbf{v}$) as the basis for matching queries to candidates is the foundation of embedding-based retrieval, recommendation systems, and attention mechanisms in transformers. *Two-vector architectures* (separate embeddings for different roles) appear beyond word2vec: query/key vectors in attention, user/item vectors in recommendation, and bi-encoder models in semantic search.

3 Dependency Parsing and Neural Network Training

3.1 What Is Dependency Parsing?

A **dependency parser** analyzes the grammatical structure of a sentence by establishing binary, asymmetric relationships between words. Each relationship is an arrow from a **head** (governor or some other term depending on your syntactic / semantic school of thought) word to a **dependent** (modifier, *idem* for the previous terminological choice comment) word. For example, in "the cat sat," "sat" is the head of the sentence (for the purposes of this class!), "cat" is its subject dependent, and "the" depends on "cat" as a determiner. A special ROOT node is added so that every word, including the main verb, is a dependent of exactly one head. The resulting structure is a tree. This contrasts with **constituency parsing**, which groups words into nested phrases (NP, VP, etc.). Dependency structures are more natural for languages with free word order and are the basis for the Universal Dependencies annotation standard used across 100+ languages. The two approaches are not mutually exclusive, but dependency parsing has become dominant in modern NLP because it pairs well with neural methods and produces structures that are easy for downstream tasks to consume.

3.2 Approaches to Dependency Parsing

There are several families of algorithms for building dependency trees. The key distinction is the trade-off between accuracy and speed. **Graph-based parsers**: These parsers treat parsing as a search over all possible trees. Given a sentence of n words, you assign a score to every possible head-dependent pair and then find the tree that maximizes the total score. The Eisner algorithm and the Chu-Liu-Edmonds algorithm are classic examples. These methods consider global structure (the entire tree at once) and can enforce well-formedness constraints, but they are slower, typically $O(n^3)$ or worse. **Transition-based parsers**: Such parsers build the tree incrementally through a sequence of local decisions. At each step, the parser looks at its current state and chooses one action (shift a word, create a left arc, or create a right arc). This process is fast, i.e. linear time $O(n)$, but each decision is made greedily without seeing the full picture. The risk is error propagation: one bad decision early on can cascade through the rest of the parse. **Neural transition-based parsers**: This final class combines the speed of transition-based parsing with the representation power of neural networks. Instead of hand-crafting indicator features, which is how traditional transition-based parsers worked, a neural network learns dense feature representations from the parser's configuration and predicts the next transition. This is the approach used in Chen and Manning (2014). Implementing a neural transition-based parsers was the primary task in Assignment 3.

3.3 Transition-Based Parsing: The Arc-Standard System

The parser maintains three data structures at every step: a **stack** σ of words currently being processed (initialized with just ROOT), a **buffer** β of words waiting to be processed (initialized with all words of the sentence in order), and a set of **dependencies** A accumulated so far (initially empty). Formally, a parser state is the triple $c = (\sigma, \beta, A)$. There are three transitions that move between states: **Shift:** Remove the first word from the buffer and push it onto the stack. Precondition: the buffer is non-empty. **Left-Arc:** The word on top of the stack becomes the head of the second-to-top word. Add the arc to A and remove the second-to-top word from the stack. Precondition: the stack has at least two items and the second-to-top item is not ROOT. **Right-Arc:** The second-to-top word on the stack becomes the head of the top word. Add the arc to A and remove the top word from the stack. Precondition: the stack has at least two items. Parsing terminates when the buffer is empty and only ROOT remains on the stack. At that point, A contains the complete dependency tree. For a sentence of n words, exactly $2n$ transitions are needed (each word is shifted once and removed by an arc once), giving the $O(n)$ runtime. **Example.** Consider the sentence "parse this sentence." The transition sequence S, S, S, LA, RA, RA produces the dependencies: (sentence, this), (parse, sentence), (ROOT, parse). The first three shifts move all words onto the stack.

The left-arc makes "sentence" the head of "this." The two right-arcs then attach "sentence" to "parse" and "parse" to ROOT.

3.4 From Features to Neural Networks

Traditional transition-based parsers used hand-engineered **indicator features**: binary flags like "is the top-of-stack word a verb?" or "does the first buffer word start with a capital letter?" These features were sparse (most are zero for any given configuration), required extensive domain expertise to design, and could not capture interactions between features without explicitly adding cross-product features, which made the feature space enormous. The Chen and Manning (2014) neural dependency parser replaced this with a simple feedforward network that takes dense, continuous representations as input. The key insight is that word embeddings already encode rich information about each word, and a neural network can learn useful feature interactions automatically through its hidden layer.

3.5 Constructing the Input

The input to the neural network is a **feature vector** extracted from the current parser configuration. This vector is a list of m token identifiers $[w_1, w_2, \ldots, w_m]$, where each w_i is the index of a specific token in the vocabulary. These tokens include items like the top three words on the stack, the first three words in the buffer, the leftmost and rightmost children of the top two stack words, and so on. There are 36 features in total in the assignment. Each token index is looked up in a pre-trained **embedding matrix** $\mathbf{E} \in \mathbb{R}^{|V| \times d}$, where $|V|$ is the vocabulary size and d is the embedding dimension. The resulting m embedding vectors are concatenated into a single input vector:

$$\mathbf{x} = [\mathbf{E}_{w_1},\ \mathbf{E}_{w_2},\ \ldots,\ \mathbf{E}_{w_m}] \in \mathbb{R}^{md}$$

With $m = 36$ features and $d = 50$-dimensional embeddings, the input is a 1800-dimensional vector. This is still much smaller and denser than the millions of sparse indicator features used in traditional parsers.

3.6 The Network Architecture

The parser uses a single-hidden-layer feedforward network. Given the concatenated input $\mathbf{x}$, the computation is:

$$\mathbf{h} = \mathrm{ReLU}(\mathbf{x}\mathbf{W} + \mathbf{b}_1) \qquad \mathbf{l} = \mathbf{h}\mathbf{U} + \mathbf{b}_2 \qquad \hat{\mathbf{y}} = \mathrm{softmax}(\mathbf{l})$$

where $\mathbf{W} \in \mathbb{R}^{md \times h}$ projects the input to a hidden layer of size h, $\mathbf{U} \in \mathbb{R}^{h \times 3}$ projects the hidden layer to 3 output classes (one per transition), and $\hat{\mathbf{y}}$ gives the predicted probability of each transition. **ReLU** (Rectified Linear Unit) is the activation function $\mathrm{ReLU}(z) = \max(z, 0)$. It introduces non-linearity, allowing the network to learn feature interactions that a linear model cannot. Without the non-linearity, stacking linear layers would collapse to a single linear transformation. ReLU is preferred over sigmoid for hidden layers because it avoids the *vanishing gradient problem*: for positive inputs, its derivative is exactly 1, so gradients flow through without shrinking. For negative inputs the derivative is 0, which can cause "dead neurons," but in practice ReLU works well and is computationally cheap.

3.7 Key PyTorch Components

`nn.Linear.` A linear layer implements the transformation $\mathbf{y} = \mathbf{x}\mathbf{W}^T + \mathbf{b}$. When you write `nn.Linear(in, out)`, PyTorch creates a weight matrix of shape $(\texttt{out}, \texttt{in})$ and a bias vector of shape $(\texttt{out},)$. The matrix multiplication projects the input from `in` dimensions down to `out` dimensions. You don't construct the weight matrix yourself. Instead, the layer manages it internally and makes it available for gradient computation. **Xavier Uniform Initialization.** The default random initialization of weight matrices can cause activations to explode or vanish as they propagate through layers. Xavier initialization draws weights from a uniform distribution scaled by the fan-in and fan-out of the layer. Fan-in is the number of inputs to a

layer (how many values come in). Fan-out is the number of outputs (how many values go out):

$$W_{ij} \sim \text{Uniform}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}},\ \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right)$$

This keeps the variance of activations roughly constant across layers, which helps the network train from the start. In PyTorch, `nn.init.xavier_uniform_(layer.weight)` applies this initialization in place. **Dropout.** During training, dropout randomly sets a fraction p of the hidden units to zero on each forward pass. This prevents the network from relying too heavily on any single neuron and acts as a form of regularization. During evaluation, dropout is turned off and all units are active. In PyTorch, calling `model.train()` enables dropout and `model.eval()` disables it. `nn.CrossEntropyLoss.` Combines softmax and cross-entropy loss into a single, numerically stable operation. Given raw logits $\mathbf{l}$ and the index of the correct class o, it computes:

$$J = -\log\left(\frac{\exp(l_o)}{\sum_i \exp(l_i)}\right) = -l_o + \log\left(\sum_i \exp(l_i)\right)$$

The second form avoids computing softmax explicitly, which prevents numerical overflow. This is why the network's forward pass returns logits (raw scores) rather than probabilities. `CrossEntropyLoss` handles the softmax internally.

3.8 The Training Loop

Training a neural network follows a four-step cycle repeated for every minibatch: **1. Forward pass.** Feed the input through the network to produce logits. In PyTorch, calling `model(input)` invokes the model's `forward` method, which chains together the embedding lookup, linear layers, ReLU, and dropout. **2. Compute loss.** Compare the predicted logits to the true labels using cross-entropy loss. The loss is a single scalar measuring how far the predictions are from the truth. **3. Backpropagation.** Calling `loss.backward()` computes the gradient of the loss with respect to every parameter in the network. PyTorch tracks all operations during the forward pass in a computational graph and walks backward through it, applying the chain rule at each node. This is the key insight of backpropagation: you don't derive gradients by hand for each layer; the chain rule decomposes the total gradient into a product of local derivatives that can be computed layer by layer. **4. Parameter update.** The optimizer (Adam in this assignment) uses the computed gradients to update each weight. Adam maintains per-parameter running averages of the gradient and its square, which gives it adaptive learning rates. Parameters that have been updated a lot get smaller steps, and vice versa. Before the next iteration, `[optimizer.zero](http://optimizer.zero)_grad()` clears the accumulated gradients so they don't carry over. **Minibatching.** Rather than processing one example at a time (slow) or the entire dataset at once (memory-intensive), we process batches of examples simultaneously. This exploits GPU parallelism and produces gradient estimates that are less noisy than single-example SGD but still computationally tractable. The assignment uses a batch size of 1024 and the minibatch dependency parsing algorithm processes multiple partial parses in parallel, feeding the current batch of parser configurations to the model and applying the predicted transitions simultaneously.

3.9 Evaluation: Unlabeled Attachment Score

The standard metric for dependency parsing is **UAS** (Unlabeled Attachment Score): the percentage of words that are assigned the correct head. "Unlabeled" means we only check whether the arc exists, not what grammatical relation it represents (subject, object, etc.). The labeled variant (LAS) additionally requires the relation label to be correct. In the assignment, the neural parser achieves over 87% UAS on the dev set after full training (about one hour). For comparison, the original Chen and Manning (2014) paper reported 92.5% UAS, and modern parsers using contextual embeddings from transformers exceed 96%.

3.10 Key Takeaways for Later Modules

- The feedforward parser is a *fixed-context* model: it sees a snapshot of the current configuration (a fixed number of stack/buffer positions) and makes a prediction. This limits what it can capture. RNNs (next module) will address this by processing sequences of arbitrary length, maintaining a hidden state that summarizes everything seen so far.
- The embedding lookup followed by a linear layer is the same pattern that appears everywhere in deep NLP. Transformers scale this idea up with *attention*, which dynamically selects which embeddings to focus on rather than concatenating a fixed set.
- Backpropagation through a computational graph is how all modern deep learning frameworks (PyTorch, TensorFlow, JAX) compute gradients. Understanding the chain rule at the level of individual layers is essential for debugging training issues, designing custom architectures, and reasoning about gradient flow problems like vanishing or exploding gradients.
- Dropout, Xavier initialization, and the Adam optimizer are standard tools in the NLP w/ Deep Learning toolkit. They recur in virtually every model from this point forward, including RNNs, transformers, and LLMs.
- The progression from sparse hand-engineered features to dense learned representations is a recurring theme in NLP's history. The same shift happened in machine translation (phrase tables to seq2seq), sentiment analysis (bag-of-words to LSTMs), and question answering (template matching to reading comprehension models). Pre-training and fine-tuning (covered in later modules) take this further by learning representations from massive unlabeled corpora.

4 Neural Machine Translation with RNNs

This section covers the core ideas from Modules 5 and 6, language models, recurrent neural networks, and sequence-to-sequence models with attention, and then walks through the implementation of a Cherokee-to-English neural machine translation (NMT) system in Assignment 4. Fun stuff!

4.1 Language Models and Recurrent Neural Networks

A **language model** assigns a probability to a sequence of words. Given a sequence $w_1, w_2, \ldots, w_T$, the model estimates:

$$P(w_1, w_2, \ldots, w_T) = \prod_{t=1}^{T} P(w_t \mid w_1, \ldots, w_{t-1})$$

Each factor conditions on the entire history. The feedforward networks from previous modules in this class, e.g. n-gram models, cannot handle this because they require a fixed-size input, i.e. one would have to decide in advance how many previous words to condition on. A **recurrent neural network** (RNN) solves this by maintaining a hidden state $\mathbf{h}_t$ that summarizes all previous inputs, updated at each time step:

$$\mathbf{h}_t = f(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b})$$

where $\mathbf{x}_t$ is the input at time t (usually a word embedding), $\mathbf{W}_h$ and $\mathbf{W}_x$ are learned weight matrices shared across all time steps, and f is a nonlinearity (typically tanh). The key property is that the same parameters are applied at every time step (**weight tying**), which allows the network to process sequences of any length. **Training** an RNN on a language modeling task means feeding it a sequence of words and, at each step, predicting the next word. The loss is the sum of cross-entropy losses at each position. Gradients are computed by **backpropagation through time** (BPTT): unrolling the recurrence across all time steps and applying the chain rule backward through the unrolled graph.

4.2 The Vanishing and Exploding Gradient Problem

When backpropagating through many time steps, gradients are multiplied by $\mathbf{W}_h$ at each step. If the largest singular value of $\mathbf{W}_h$ is less than 1, gradients shrink exponentially and the network cannot learn long-range dependencies (**vanishing gradients**). If the largest singular value exceeds 1, gradients grow exponentially (**exploding gradients**), causing numerical instability. **Gradient clipping** addresses the exploding case: if $\|\mathbf{g}\| > \theta$ for some threshold θ, rescale $\mathbf{g} \leftarrow \frac{\theta}{\|\mathbf{g}\|}\mathbf{g}$. This caps the gradient magnitude without changing its direction. It is simple, effective, and used in virtually every RNN training pipeline. The vanishing case is harder to patch and motivates architectural changes, specifically the LSTM.

4.3 LSTMs: Long Short-Term Memory

An **LSTM** replaces the simple hidden state update with a gated architecture that controls what information flows in, what is retained, and what is output. At each time step t, an LSTM maintains two vectors: a **hidden state** $\mathbf{h}_t$ and a **cell state** $\mathbf{c}_t$. The gates and updates are:

$$\begin{aligned}
\mathbf{f}_t &= \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) && \text{(forget gate)} \\
\mathbf{i}_t &= \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) && \text{(input gate)} \\
\widetilde{\mathbf{c}}_t &= \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) && \text{(candidate cell)} \\
\mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \widetilde{\mathbf{c}}_t && \text{(cell update)} \\
\mathbf{o}_t &= \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) && \text{(output gate)} \\
\mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t) && \text{(hidden state)}
\end{aligned}$$

where σ is the sigmoid function and $\odot$ is element-wise multiplication. **Hidden state vs. cell state** is one of the most important distinctions to internalize. The **cell state** $\mathbf{c}_t$ is the long-term memory. It

flows through time with only element-wise operations (multiply by forget gate, add new input), which means gradients can propagate through it without repeated matrix multiplications. This is what solves the vanishing gradient problem. The **hidden state** $\mathbf{h}_t$ is the working output at each step, a filtered version of the cell state that gets passed to the next layer or used for predictions. In the seq2seq model we implement, both are needed: the encoder passes both $\mathbf{h}$ and $\mathbf{c}$ to the decoder so that it receives the full learned representation. **Why gates work:** Each gate is a sigmoid, outputting values in $(0, 1)$ that act as soft switches. The forget gate decides which dimensions of the previous cell to keep. The input gate decides which dimensions of the new candidate to write. The output gate decides which dimensions of the cell to expose. Because these are learned, the network can discover on its own which information to remember over long distances and which to discard.

4.4 Bidirectional LSTMs

A standard LSTM processes a sequence left to right, so $\mathbf{h}_t$ only captures information from $w_1, \ldots, w_t$. For many tasks (including machine translation), the meaning of a word depends on both its left and right context. A **bidirectional LSTM** runs two LSTMs in parallel: one forward ($\overrightarrow{\mathbf{h}}_t$) and one backward ($\overleftarrow{\mathbf{h}}_t$). Their outputs are concatenated:

$$\mathbf{h}_t = [\overrightarrow{\mathbf{h}}_t;\ \overleftarrow{\mathbf{h}}_t] \in \mathbb{R}^{2h}$$

where h is the hidden size of each direction. This gives the model access to the full sentence context at every position. The trade-off is that bidirectional models cannot be used autoregressively (you cannot condition on future words during generation), so they are used for encoding but not decoding.

4.5 Sequence-to-Sequence Architecture

Neural machine translation (NMT) frames translation as a sequence-to-sequence problem: given a source sentence, generate a target sentence. The standard architecture has two components. The **encoder** reads the entire source sentence and compresses it into a sequence of hidden states. In Assignment 4, the encoder is a bidirectional LSTM. It produces hidden states $\mathbf{h}_1^{\text{enc}}, \ldots, \mathbf{h}_m^{\text{enc}} \in \mathbb{R}^{2h}$ (one per source word) and a final state that initializes the decoder. Because the encoder is bidirectional, its hidden size is $2h$, but the decoder is unidirectional with hidden size h. **Projection layers** bridge this gap: linear transformations $\mathbf{W}_h \in \mathbb{R}^{h \times 2h}$ and $\mathbf{W}_c \in \mathbb{R}^{h \times 2h}$ (with no bias) map the concatenated forward and backward final states down to the decoder's dimension:

$$\mathbf{h}_0^{\text{dec}} = \mathbf{W}_h[\overrightarrow{\mathbf{h}}_m;\ \overleftarrow{\mathbf{h}}_1] \qquad \mathbf{c}_0^{\text{dec}} = \mathbf{W}_c[\overrightarrow{\mathbf{c}}_m;\ \overleftarrow{\mathbf{c}}_1]$$

The decoder needs both the hidden state and the cell state from the encoder to receive the full information the LSTM has learned. The **decoder** is a unidirectional LSTMCell that generates the target sentence one word at a time, conditioned on the encoder's output and all previously generated words. At each step, it receives the concatenation of the current target word embedding and the previous attention output, then produces a new hidden state from which the next word is predicted.

4.6 Attention Mechanism

Without attention, the decoder must generate the entire target sentence from a single fixed-length vector (the encoder's final state). This is a severe information bottleneck, especially for long sentences. **Attention** (Bahdanau et al., 2015; Luong et al., 2015) allows the decoder to look back at all encoder hidden states at every decoding step and dynamically focus on the most relevant parts of the source sentence. The attention computation at each decoder time step t proceeds as follows. Let $\mathbf{h}_t^{\text{dec}} \in \mathbb{R}^h$ be the current decoder hidden state and $\mathbf{H}^{\text{enc}} \in \mathbb{R}^{b \times m \times 2h}$ be the matrix of all encoder hidden states (for a batch of b sentences, each of length m). **Step 1: Compute attention scores.** Project the encoder hidden states once (this is done outside the decoding loop for efficiency):

$$\mathbf{H}^{\text{proj}} = \mathbf{H}^{\text{enc}}\mathbf{W}_{\text{att}}^{\top} \in \mathbb{R}^{b \times m \times h}$$

Then compute the alignment score between the decoder state and each encoder state via batched matrix multiplication:

$$\mathbf{e}_t = \mathbf{H}^{\text{proj}}\,\mathbf{h}_t^{\text{dec}} \in \mathbb{R}^{b\times m}$$

This is a **multiplicative (Luong) attention** score: each entry $e_{t,i}$ measures how relevant source position i is to the current decoding step. **Step 2: Mask and normalize.** Before applying softmax, positions corresponding to padding tokens in the source sentence are set to $-\infty$. This ensures that after softmax, padding positions receive zero attention weight and do not influence the output. The attention weights are:

$$\boldsymbol{\alpha}_t = \text{softmax}(\mathbf{e}_t) \in \mathbb{R}^{b\times m}$$

Step 3: Compute context vector. The context vector is a weighted sum of encoder hidden states:

$$\mathbf{a}_t = \boldsymbol{\alpha}_t\,\mathbf{H}^{\text{enc}} \in \mathbb{R}^{b\times 2h}$$

Step 4: Combine and project. Concatenate the decoder hidden state with the context vector, project down, and apply dropout:

$$\mathbf{U}_t = [\mathbf{h}_t^{\text{dec}};\ \mathbf{a}_t] \in \mathbb{R}^{3h} \qquad \mathbf{V}_t = \mathbf{W}_u\mathbf{U}_t \in \mathbb{R}^{h} \qquad \mathbf{O}_t = \text{dropout}(\tanh(\mathbf{V}_t))$$

The combined output $\mathbf{O}_t$ is then projected to the target vocabulary size to produce logits for the next word prediction. $\mathbf{O}_t$ is also fed back as input to the next decoder step (concatenated with the next target embedding), giving the decoder a direct channel of attention information across steps. **Types of attention.** The assignment uses multiplicative (dot-product) attention. Other variants include **additive attention** (Bahdanau), which uses a feedforward network to compute scores: $e_{t,i} = \mathbf{v}^\top \tanh(\mathbf{W}_1\mathbf{h}_i^{\text{enc}} + \mathbf{W}_2\mathbf{h}_t^{\text{dec}})$. Additive attention is more expressive but slower; multiplicative attention is simpler and pairs well with GPU-optimized batched matrix multiplication.

4.7 Implementation Details: What We Built and Why

4.7.1 Sentence Padding

Neural networks require fixed-size tensors, but sentences vary in length. The `pad_sents` function pads all sentences in a batch to the length of the longest sentence using a special `<pad>` token. This is the simplest approach; the key subtlety is that padded positions must be masked out in the attention computation (set to $-\infty$ before softmax) so the model does not attend to meaningless padding.

4.7.2 Embedding Layers

Source and target languages each get their own `nn.Embedding` layer. The `padding_idx` argument tells PyTorch to keep the embedding vector for the pad token as all zeros and exclude it from gradient updates. This matters because if the pad embedding were nonzero, it would contribute noise to packed sequences and attention computations.

4.7.3 The Encoder: Pack, Run, Unpack

Variable-length sequences in a batch create a problem: shorter sentences have padding that we don't want the LSTM to process. PyTorch's `pack_padded_sequence` compresses the batch by removing padding, and `pad_packed_sequence` restores the original shape afterward. The sequence of operations is: embed the source tokens, pack them, run the bidirectional LSTM, unpack, and permute the output from $(m, b, 2h)$ to $(b, m, 2h)$. The permutation is needed because PyTorch's LSTM outputs time-first ordering, but the rest of the model expects batch-first. After running the LSTM, the final hidden and cell states from the forward and backward directions are concatenated and projected down from $2h$ to h dimensions using learned linear layers (no bias). This is the initialization for the decoder.

4.7.4 The Decoder Loop

The decoder processes one target time step at a time. At each step, it concatenates the current target word embedding with the previous attention output $\mathbf{O}_{t-1}$, feeds this through an LSTMCell (not a full LSTM as the loop is explicit so we can inject attention at each step), computes attention over the encoder states, and produces the new attention output $\mathbf{O}_t$. The outputs from all time steps are stacked into a tensor and projected to vocabulary-size logits. A question that arose during implementation: why is the decoder an `LSTMCell` rather than an `nn.LSTM`? Because the decoder's input at each step depends on the attention output from the previous step, which means we cannot run all steps in a single call. The explicit loop with an LSTMCell gives us the control to inject attention feedback at each iteration.

4.7.5 Tensor Dimension Manipulation

Working with batched attention requires careful use of `squeeze`, `unsqueeze`, and `torch.bmm`. `squeeze(dim)` removes a dimension of size 1; `unsqueeze(dim)` adds one. These are needed to match the shapes required by `torch.bmm` (batched matrix multiplication), which expects 3D tensors of shape (b, n, m) and (b, m, p). For example, computing attention scores requires the decoder hidden state to be reshaped from (b, h) to $(b, h, 1)$ via `unsqueeze(2)`, and the result squeezed back from $(b, m, 1)$ to (b, m). Getting these shapes right is one of the most error-prone parts of implementing attention from scratch.

4.8 BLEU Score

BLEU (Bilingual Evaluation Understudy) is the standard automatic metric for machine translation. It measures the overlap between a machine-generated translation and one or more reference translations. BLEU computes a **modified n-gram precision** for $n = 1, 2, 3, 4$. For each n-gram in the candidate translation, it checks whether that n-gram appears in any reference, but clips the count so that no n-gram gets more credit than the maximum number of times it appears in any single reference. This prevents a degenerate translation like "the the the the" from scoring well against a reference containing "the." The individual precisions p_1, p_2, p_3, p_4 are combined via a geometric mean (equivalent to averaging their logs), and a **brevity penalty** (BP) penalizes translations that are shorter than the reference:

$$\mathrm{BP} = \begin{cases} 1 & \text{if } c > r \\ \exp(1 - r/c) & \text{if } c \leq r \end{cases} \qquad \mathrm{BLEU} = \mathrm{BP} \cdot \exp\left(\sum_{n=1}^{4} \frac{1}{4} \log p_n \right)$$

where c is the candidate length and r is the (effective) reference length. **Key properties:** BLEU ranges from 0 to 1 (often reported as 0–100). It correlates reasonably with human judgment at the corpus level but can be misleading for individual sentences. Having multiple references generally increases the score because there are more valid n-grams to match. A BLEU score of 12.13 (what we achieved for Cherokee-to-English) is modest, reflecting the difficulty of translating a low-resource language with limited training data.

4.9 Beam Search

Greedy decoding picks the highest-probability word at each step, which can lead to suboptimal translations because a locally good choice might preclude a globally better sequence. **Beam search** maintains the top k partial hypotheses (the beam width) at each step. At each time step, every hypothesis is extended by every possible next word, the resulting candidates are scored, and only the top k survive. This explores a larger portion of the search space without the exponential cost of exhaustive search. In practice, beam widths of 5–10 are typical; larger beams give diminishing returns.

4.10 Engineering Takeaways

Attention is the precursor to transformers. The attention mechanism implemented here, computing alignment scores, normalizing with softmax, taking a weighted sum of value vectors, is exactly the same computation that happens inside every transformer layer's self-attention. The difference is that transformers apply attention in parallel across all positions (no recurrence), which enables massive parallelization. Understanding attention in the RNN context makes the jump to transformers much more intuitive. *Hidden state vs. cell state matters in practice.* When using LSTMs in production systems, you need to pass both $\mathbf{h}$ and $\mathbf{c}$ when transferring state between components (e.g., from encoder to decoder, or when saving and restoring state for streaming applications). Forgetting the cell state is a common bug that silently degrades performance. *Projection layers are dimensionality adapters.* Whenever two components have mismatched dimensions (bidirectional encoder outputting $2h$ feeding into a unidirectional decoder expecting h), a learned linear projection bridges the gap. This is a pattern that appears throughout deep learning: in residual connections with mismatched dimensions, in multi-head attention where heads are projected back to model dimension, and in adapter layers for fine-tuning. *Masking is essential for variable-length sequences.* Any system that processes batches of sequences with different lengths must mask out padding. In attention, unmasked padding tokens receive nonzero attention weight and corrupt the context vector. In loss computation, unmasked padding inflates the loss. Getting masking right is one of the most common sources of subtle bugs in production NLP systems. *Pack/unpack for RNNs vs. masking for transformers.* PyTorch's `pack_padded_sequence` / `pad_packed_sequence` is the RNN-specific solution to variable-length batching. Transformers use attention masks instead, which are simpler to implement but serve the same purpose. Both approaches avoid wasting computation on padding tokens. *Squeeze and unsqueeze are the "glue" of tensor programming.* Most dimension errors in PyTorch come from mismatched shapes in matrix multiplications. Developing an intuition for when to add or remove a dimension, and understanding that `torch.bmm` requires exactly 3D inputs, saves significant debugging time. When implementing attention or any operation that mixes batch and sequence dimensions, it helps to annotate each tensor's shape in comments. *BLEU is useful but limited.* In production, BLEU is one signal among many. It rewards exact n-gram matches and penalizes valid paraphrases. Modern evaluation increasingly uses learned metrics (BLEURT, COMET) or human evaluation. However, BLEU remains the standard baseline metric because it is fast, deterministic, and well-understood. *The seq2seq pattern generalizes beyond translation.* Encoder-decoder architectures are used for summarization, code generation, speech recognition, and any task that maps one sequence to another. T5 ("Text-to-Text Transfer Transformer") made this explicit by framing every NLP task as sequence-to-sequence. Understanding the encoder-decoder pattern and attention mechanism from this assignment provides the foundation for working with these models.

5 Self-Attention, Transformers, and Pretraining

This section covers the core ideas from Modules 7 and 8: self-attention, the Transformer architecture, and the pretrain-then-finetune paradigm that powers essentially all of modern NLP. Assignment 5 walks through these at the implementation level, building a character-level Transformer from a forked `minGPT` codebase, pretraining it on Wikipedia text with a span-corruption objective, and finetuning it to answer questions like "Where was [person] born?" The dramatic accuracy lift from pretraining (1.2% → 33.6%) is the tangible payoff and the empirical lesson at the heart of the assignment.

5.1 From RNN Attention to Self-Attention

The attention mechanism in Section 4.6 was already a generalized lookup: the decoder at each time step computed alignment scores against every encoder state and produced a weighted sum of value vectors. **Self-attention** applies the same operation, with two crucial differences:

1. **Within a single sequence rather than across an encoder-decoder boundary.** Every token in the input produces its own query, key, and value. Each token can therefore "look at" every other token, including itself.
2. **Parallel rather than sequential.** No recurrence, every position computes its query and attends to all keys in parallel. This is what makes Transformers vastly faster to train than RNNs on modern hardware: there's no $\mathbf{h}_t$ waiting on $\mathbf{h}_{t-1}$.

Conceptually, the math is the same as in NMT attention, dot-product similarity, softmax normalization, weighted sum of values. But the architectural implications differ enormously: by removing recurrence, the entire sequence is processed in one matrix multiply, exposing massive parallelism that GPUs exploit.

5.2 The Self-Attention Layer

Given an input $\mathbf{X} \in \mathbb{R}^{T \times d}$ (one row per token, sequence length T, hidden dimension d), compute three projections:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V$$

where $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d \times d}$ are learned weight matrices. Each row of $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ is the query, key, and value vector for the corresponding token. **Each token simultaneously plays three roles**: it asks (query), it advertises (key), and it carries content (value). The attention output is:

$$\mathbf{H} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V}$$

The $\mathbf{Q}\mathbf{K}^\top$ matrix is $T \times T$, with entry (i, j) being token i's query dotted with token j's key, a measure of how relevant token j is to what token i is looking for. The softmax operates rowe-wise, turning each row into a probability distribution $\boldsymbol{\alpha}_i$ over the T positions. Multiplying by $\mathbf{V}$ produces a weighted average of value vectors for each row, giving each token a **contextualized representation** that blends in information from elsewhere. **Each row of $\mathbf{H}$** is exactly the part-(a)-style output $c = \sum_i v_i \alpha_i$ from the attention exploration, but computed for every token's perspective in parallel. The full matrix multiplication is just T attention lookups stacked. **The $1/\sqrt{d}$ scaling** is a numerical fix, not a conceptual one. Without it, dot products of two random d-dimensional vectors grow with $\sqrt{d}$, pushing softmax toward saturated regions where gradients vanish. Dividing by $\sqrt{d}$ keeps the input scale to softmax roughly constant.

5.3 Multi-Headed Attention

A single attention head has surprising failure modes. Section 3 of A5 made this explicit: when one key vector $\mathbf{k}_a$ has variable magnitude (even with the correct direction), a single-head construction designed to

average two values can degenerate. Sometimes the output is the desired average, sometimes it collapses to one of the values alone. The softmax-mediated balancing of two scores is fragile under perturbation. **Multi-headed attention** sidesteps this by running multiple attention operations in parallel, each with its own learned projections, then concatenating their outputs:

$$\text{head}_i = \text{softmax}\left(\frac{\mathbf{Q}_i\mathbf{K}_i^\top}{\sqrt{d_h}}\right)\mathbf{V}_i, \quad \text{MultiHead}(\mathbf{X}) = [\text{head}_1; \ldots; \text{head}_H]\mathbf{W}_O$$

where each head operates on a slice of the embedding (per-head dimension $d_h = d/H$), and $\mathbf{W}_O$ is a final output projection. **Why it works:** each head can do a single-peak, "copy-style" lookup, which is the *robust* regime of attention. The averaging across multiple values happens at the *output combination* step, not inside the softmax. Different heads can also learn to attend to different things in parallel, one head tracking syntactic dependencies, another tracking coreference, another tracking semantic similarity, and so on. In A5 Part 3, we proved this rigorously: if you design two heads $\mathbf{q}_1 = \beta\boldsymbol{\mu}_a$ and $\mathbf{q}_2 = \beta\boldsymbol{\mu}_b$, head 1 reliably copies $\mathbf{v}_a$ and head 2 reliably copies $\mathbf{v}_b$, regardless of magnitude perturbations on $\mathbf{k}_a$. Their average gives $\frac{1}{2}(\mathbf{v}_a + \mathbf{v}_b)$ with low variance, the single-head construction's reliability problem is gone.

5.4 The Transformer Block

A Transformer is a stack of identical blocks. Each block has two main components:

1. **Multi-headed self-attention** (with residual connection and layer normalization)
2. **Position-wise feed-forward network** (a 2-layer MLP applied independently to each token)

In abstract form:

$$\mathbf{H} = \text{LayerNorm}(\mathbf{X} + \text{MultiHead}(\mathbf{X}))$$
$$\mathbf{Z} = \text{LayerNorm}(\mathbf{H} + \text{FFN}(\mathbf{H}))$$

where $\text{FFN}(\mathbf{h}) = \mathbf{W}_2 \cdot \text{ReLU}(\mathbf{W}_1\mathbf{h} + \mathbf{b}_1) + \mathbf{b}_2$. **The two layers do complementary work.** Attention mixes information across positions, every token sees every other token. The FFN reprocesses each position independently, no cross-token mixing, just a per-token nonlinear transformation. The two alternate, layer after layer. Stacking many of these (4 in our toy assignment, 12 in BERT-base, 96 in GPT-3, hundreds in frontier models) lets the model build increasingly abstract representations layer by layer. Lower layers tend to capture surface patterns; higher layers capture syntax, semantics, and world knowledge.

5.5 The Permutation Problem

A subtle but critical observation: **self-attention is permutation-equivariant**. Permute the input tokens and the output is just the same shuffle of the original output. Concretely, if $\mathbf{X}_{\text{perm}} = \mathbf{PX}$, then $\mathbf{Z}_{\text{perm}} = \mathbf{PZ}$. The proof (Section 4 of A5) chains together two given identities, $\text{softmax}(\mathbf{PAP}^\top) = \mathbf{P}\,\text{softmax}(\mathbf{A})\mathbf{P}^\top$ and $\text{ReLU}(\mathbf{PA}) = \mathbf{P}\,\text{ReLU}(\mathbf{A})$, with two structural facts: $\mathbf{P}^\top\mathbf{P} = \mathbf{I}$ (permutations are orthogonal), and $\mathbf{P} \cdot (\mathbf{1} \cdot \mathbf{b}) = \mathbf{1} \cdot \mathbf{b}$ (permuting rows of a constant matrix leaves it unchanged). The result is that nothing in the architecture privileges position over identity. **Why this is fatal for text:** word order carries meaning. "Dog bites man" $\neq$ "Man bites dog." A permutation-equivariant model is essentially a bag-of-words classifier, it sees *what* tokens appeared but not *in what order*. It can do topic detection and similar shallow tasks, but it cannot capture syntax or any structure that depends on sequencing.

5.6 Position Embeddings: The Fix

Add a position-dependent vector $\boldsymbol{\Phi}_t$ to each token's embedding before feeding into the network: $\mathbf{X}_{\text{pos}} = \mathbf{X} + \boldsymbol{\Phi}$. The key property: $\boldsymbol{\Phi}$ depends only on position t, not on the token identity. Now permuting $\mathbf{X}$ breaks the equivariance:

$$\mathbf{PX} + \boldsymbol{\Phi} \neq \mathbf{P}(\mathbf{X} + \boldsymbol{\Phi})$$

unless $\mathbf{\Phi} = \mathbf{P}\mathbf{\Phi}$, which only holds for the identity permutation. The same word at different positions becomes a different input to the model. There are two main families of position embeddings: **Learned absolute embeddings.** A separate vector per position, treated as parameters and trained. Used in the original Transformer paper and BERT. Simple but inflexible: the model only knows about positions it saw during training. **Sinusoidal embeddings.** A fixed (non-learned) function of position:

$$\mathbf{\Phi}_{(t,2i)} = \sin(t/10000^{2i/d}), \quad \mathbf{\Phi}_{(t,2i+1)} = \cos(t/10000^{2i/d})$$

Frequencies span a geometric range, at $i = 0$ the period is 2π, at $i = d/2-1$ it's roughly $2\pi \cdot 10000$. Different dimensions oscillate at different rates, giving the network a multi-scale "ruler" for position. **Sinusoidal embeddings are guaranteed unique per position.** This follows from the irrationality of 2π: at $i = 0$, two integer positions $t_1 \neq t_2$ would need to differ by a multiple of 2π to share an embedding, but no nonzero integer is a multiple of 2π. The lowest-frequency component alone distinguishes every position. Both approaches are *absolute*: they encode "position 5" rather than "5 positions away." Models trained on length-128 sequences can struggle to extrapolate to length-1024 because the corresponding position embeddings come from distributions never seen in training.

5.7 Rotary Position Embeddings (RoPE)

RoPE solves the extrapolation problem with a rotational scheme. Instead of *adding* $\mathbf{\Phi}$ to $\mathbf{X}$, RoPE *multiplies* query and key vectors by a position-dependent rotation matrix, applied per-head, just before attention scores are computed. For a 2D feature pair $(x^{(1)}, x^{(2)})$ at position t, RoPE rotates by an angle $t\theta$:

$$\mathrm{RoPE}((x^{(1)}, x^{(2)}), t) = \begin{bmatrix} \cos(t\theta) & -\sin(t\theta) \\ \sin(t\theta) & \cos(t\theta) \end{bmatrix} \begin{bmatrix} x^{(1)} \\ x^{(2)} \end{bmatrix}$$

For higher-dimensional features, the same rotation is applied to consecutive pairs of dimensions, each with its own frequency $\theta_i = 10000^{-2(i-1)/d}$. RoPE is applied to $\mathbf{Q}$ and $\mathbf{K}$ but **not** to $\mathbf{V}$, since values carry content rather than addressing information. **The key property:** the dot product after rotation depends only on relative position. For complex-number-valued features:

$$\langle \mathrm{RoPE}(\mathbf{z}_1, t_1), \mathrm{RoPE}(\mathbf{z}_2, t_2) \rangle = \langle \mathrm{RoPE}(\mathbf{z}_1, t_1 - t_2), \mathrm{RoPE}(\mathbf{z}_2, 0) \rangle$$

So query at position t_1 matched against key at position t_2 produces a score that depends only on the offset $t_1 - t_2$, not the absolute positions. This is **relative position encoding** in disguise, and it's why models trained on shorter sequences can generalize better to longer ones, only relative offsets matter. **Implementation trick.** Pairs of features can be reinterpreted as complex numbers, and the rotation becomes a complex multiplication by $e^{it\theta} = \cos(t\theta) + i\sin(t\theta)$. PyTorch's `view_as_complex` and `view_as_real` make this a couple of lines: reshape the last dim from h_s to $(h_s/2, 2)$, view as complex, multiply element-wise by the precomputed rotation cache, view as real, reshape back. **Empirical impact in A5.** Swapping vanilla learned positional embeddings for RoPE jumped dev accuracy from 19.0% to 33.6% on the birth-place task, a substantial gain on the same architecture and same training data. This kind of architectural detail is exactly why modern LLMs (Llama, Mistral, Gemma, etc.) almost universally use RoPE rather than learned or sinusoidal embeddings.

5.8 Pretraining via Span Corruption

The pretrain-finetune paradigm splits training into two phases:

1. **Pretrain** on a massive unlabeled corpus with a self-supervised objective.
2. **Finetune** on a small labeled dataset for a specific downstream task.

A5 used a simplified version of T5's **span corruption** objective. Given a document, randomly mask a contiguous span and ask the model to reconstruct it. Concretely, take a document like "Khatchig Mouradian. Khatchig Mouradian is a journalist, writer..." and:

1. Truncate to a random length $L \in [4, \frac{3}{4}\texttt{block_size}]$.
2. Pick a random masked-content length $M \sim \text{Uniform}[1, L/2]$.
3. Pick a random prefix length P, splitting the document into prefix / masked_content / suffix.
4. Rearrange as `[prefix] ??  [suffix] ??  [masked_content] ??  [pads]`.

The model sees the rearranged version and must autoregressively predict the next character, including reconstructing the masked content from the surrounding context. **Why this teaches the model anything useful.** To predict the masked span correctly, the model must learn the statistical structure of language: vocabulary patterns, syntax, common factual associations, named-entity patterns. After 650 epochs over 418K characters of Wikipedia text, those patterns are baked into the weights as **parametric memory**, the model has implicitly memorized "Khatchig Mouradian → journalist," "born in Lebanon," and countless other small facts encountered in the corpus.

5.9 Fine-Tuning on Downstream Tasks

After pretraining, the model is finetuned on a specific labeled task. In A5, the task was birth-place prediction: given `"Where was [name] born???"`, predict the city. The architecture doesn't change. The same model that was trained to reconstruct masked spans is now trained on (question, answer) pairs. **Crucially, the weights are not reinitialized**, finetuning continues from the pretrained checkpoint. The model accesses the world knowledge it absorbed during pretraining to answer specific questions. **Hyperparameter signal of the difference between phases:**

- Pretraining: 650 epochs, learning rate 6×10^{-3}, large dataset.
- Finetuning: 10 epochs, learning rate 6×10^{-4}, small dataset.

The intuition: the model is mostly already correct after pretraining; finetuning is a small adjustment to specialize for the new task format. Higher learning rates and many epochs would risk catastrophically overwriting the pretrained knowledge.

5.10 The Pretraining Payoff: Knowledge Access Experiment

A5's controlled experiment quantifies the impact of pretraining:

Configuration	Dev accuracy on birth-place
Vanilla, no pretraining	1.2%
Vanilla + sinusoidal pretraining	19.0%
Vanilla + RoPE pretraining	33.6%

The "always say London" baseline gets about 5%. **The 16x improvement from pretraining alone** (1.2% → 19.0%) is the key empirical lesson. The model is the same size, sees the same labeled finetuning data, and uses identical training code, what changes is whether it has seen Wikipedia first. **The interpretation.** The 2,000-pair finetuning set is far too small to teach a 3.3M-parameter Transformer global facts about people and places. It can barely learn "answer in city-format" without pretraining, let alone *which* cities go with *which* names. But pretraining on 418K characters of Wikipedia exposes the model to those facts in their natural distributional form ("Khatchig Mouradian is a journalist born in..."). Finetuning

then teaches the model to *expose* this knowledge in question-answer format. **This is the modern LLM playbook in miniature.** GPT-4, Claude, Gemini, and Llama are trained on trillions of tokens (vs. 418K characters here) and then fine-tuned for instruction-following (vs. 2K Q&A pairs here). The exact same paradigm, scaled by 5–6 orders of magnitude.

5.11 The Model We Actually Built

For reference, the A5 architecture has these specs:

- **Vocab size:** 256 (character-level, ASCII)
- **Block size:** 128 characters
- **Embedding dim:** 256
- **Layers:** 4
- **Heads:** 8 (per layer)
- **Per-head dim:** 32
- **Total parameters:** 3.3M (vanilla) or 3.29M (RoPE, drops the 32K-element learned position embedding)

The RoPE variant has *fewer* parameters than the vanilla variant, yet achieves nearly double the dev accuracy. **Architectural choices outweigh parameter count at this scale.** Pretraining on a single Azure GPU completed in ∼15 minutes (650 epochs over ∼6K wiki lines, batch size 128). Finetuning was much faster, about 2 minutes for 10 epochs. The development cycle "edit code → push to VM → train → evaluate" is a microcosm of how production NLP work actually happens.

5.12 Engineering Takeaways

Self-attention is the foundation of modern NLP. Every major model since the original "Attention Is All You Need" (2017), BERT, GPT, T5, Llama, Mistral, Claude, Gemini, is built on the Transformer block. Understanding the **Q/K/V** mechanism is essential vocabulary for the rest of one's career in this space. The "soft, content-based lookup" mental model generalizes from language to vision (ViT), reinforcement learning, protein folding (AlphaFold), code generation, robotics, and multimodal models.

Position embeddings are non-obvious but essential. Without them, attention is permutation-invariant, a bag-of-words classifier. The choice of scheme matters more than expected: RoPE has displaced both learned and sinusoidal embeddings in current frontier models because it gives true relative position encoding and extrapolates to longer sequences than seen in training.

Multi-head attention is the engineering solution to single-head fragility. The Section 3 analysis in A5 is a rare clean mathematical proof of why a hand-tweakable property of one head fails under perturbation but multiple heads do not. The lesson generalizes: when one parametric mechanism is fragile, run several in parallel and average (or concatenate, then project).

Pretraining is the central insight that powers modern ML. The 16x performance gap between no-pretrain and pretrain in A5 is the entire reason the field has organized itself around foundation models. Models access **parametric memory**, knowledge encoded into weights during pretraining, and fine-tuning is just specialization. This has reshaped the economics of NLP: pretraining costs millions of dollars but is amortized across thousands of downstream applications.

Span corruption / masked language modeling are foundational pretraining objectives. Different variants (BERT's random tokens, T5's spans, GPT's causal LM) all exploit the same insight: predicting hidden

tokens forces the model to learn distributional structure, which is enough to extract surprising amounts of world knowledge. The objective is self-supervised (no labels needed), which is why it scales.

The transformer block (attention + FFN + residual + layer norm) is the universal NLP architecture. Same building block for encoders (BERT), decoders (GPT-style LMs), and encoder-decoders (T5, NLLB). Stacking 4 (toy), 12 (BERT-base), 96 (GPT-3), or 100+ (frontier models) layers gives different capability tiers. Practitioners spend remarkably little time questioning the block; almost all innovation happens *around* it (training objectives, scaling, alignment, positional encoding, attention efficiency).

Tensor shape gymnastics are where most bugs live. Self-attention requires careful (B, nh, T, h_s) reshaping, transposes, and matrix multiplications. The bridge between mathematical formulas and PyTorch tensors is one of the most error-prone parts of implementation. Annotating shapes in comments and using `view_as_complex` / `view_as_real` for RoPE saved real time in A5.

Architecture details outpunch model size at the small-model end. Going from learned position embeddings to RoPE in A5, about 50 lines of code, no extra parameters, nearly doubled dev accuracy on the birth-place task. This is the kind of finding that motivates the small army of researchers tweaking attention variants, normalization placements, activation functions, and so on, even at the frontier.

The pretrain → finetune paradigm has become pretrain → finetune → align. Modern LLMs add ideas A5 didn't cover: instruction-tuning, RLHF (Reinforcement Learning from Human Feedback), DPO, constitutional AI methods, and so on. But all of these are *layers on top of* the pretrain-then-finetune scaffold A5 demonstrated. Understanding A5 puts you within striking distance of understanding any modern LLM training pipeline.

Concerns about reliability, bias, and hallucination scale with deployment. Section 2 of the assignment (not graded) raised them explicitly: a pretrained model "retrieves" answers from parametric memory, but it's impossible to tell from the output whether the model genuinely knows or is making something up. It also has no way to express "I don't know," and its biases reflect its training corpus. These concerns motivate retrieval-augmented generation (RAG), uncertainty calibration, refusal training, and citation-aware generation, all active areas of NLP today.

Looking forward. From this foundation, the rest of modern NLP unfolds: scaling laws (Kaplan et al., Chinchilla), efficient attention (FlashAttention, sliding window), mixture-of-experts (MoE), in-context learning, chain-of-thought prompting, tool use, agentic systems. None of these are conceptually orthogonal to A5's content; they're all extensions and engineering optimizations on top of the same Transformer foundation. The shift in research focus from "design new architectures" to "scale, align, and apply the Transformer" is one of the defining characteristics of the late-2020s NLP era.